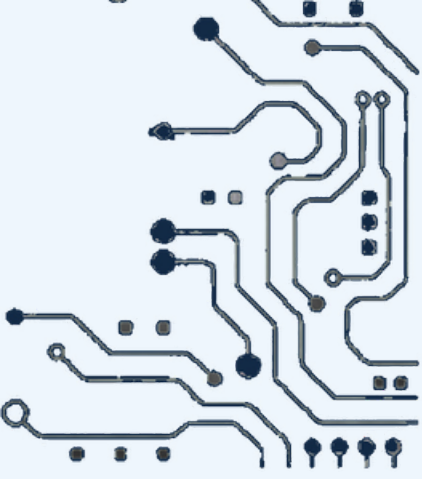




E-TRONIX'26

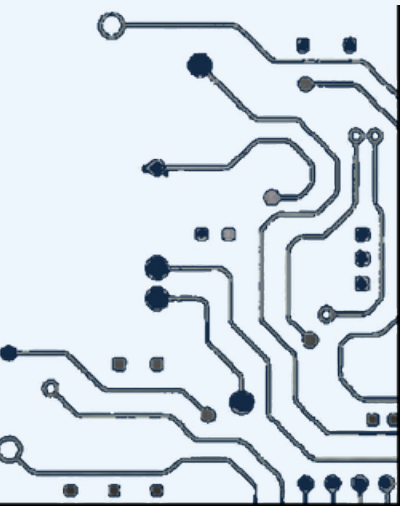
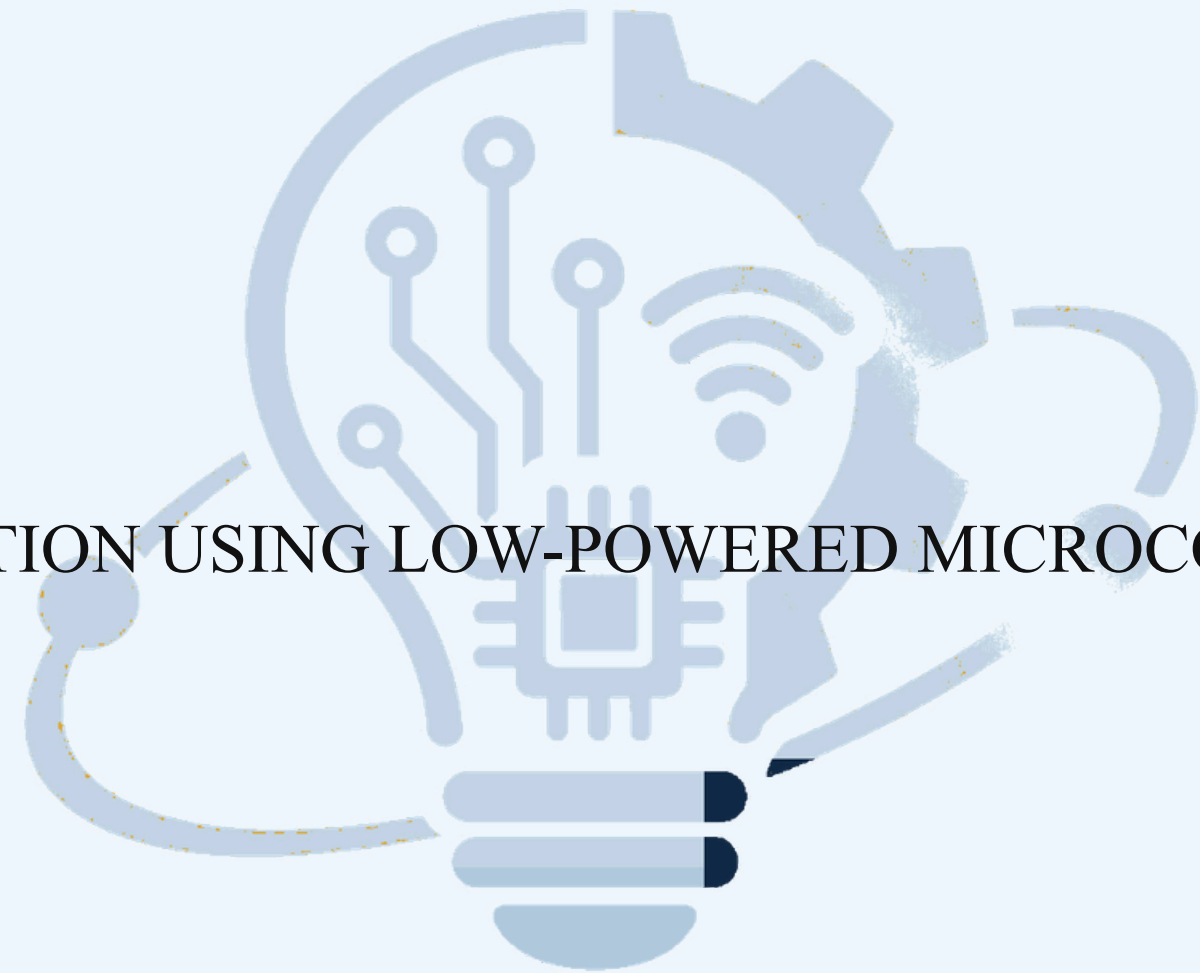


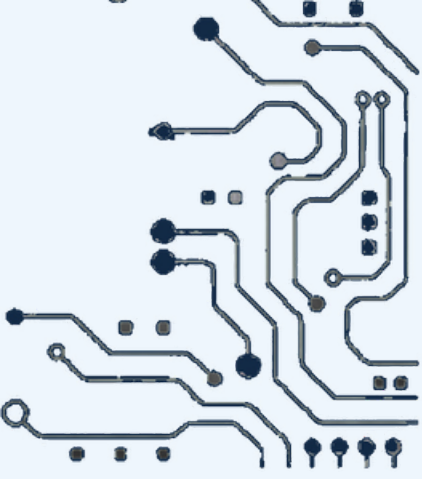
TEAM NAME : NEXGEN

PROBELM STATEMENT NO : PSE02

PROBLEM STATEMENT TITLE: WAKE WORD DETECTION USING LOW-POWERED MICROCONTROLLER

PS Category - HARDWARE





E-TRONIX'26



REAL WORLD PROBLEM :

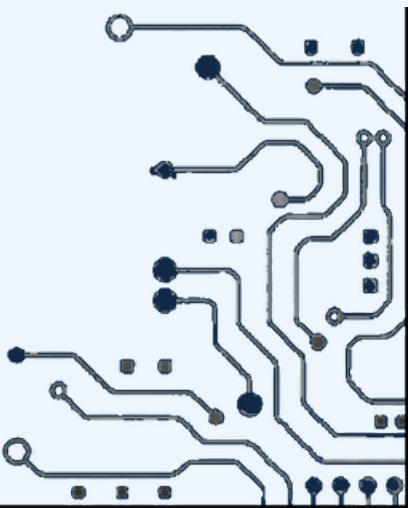
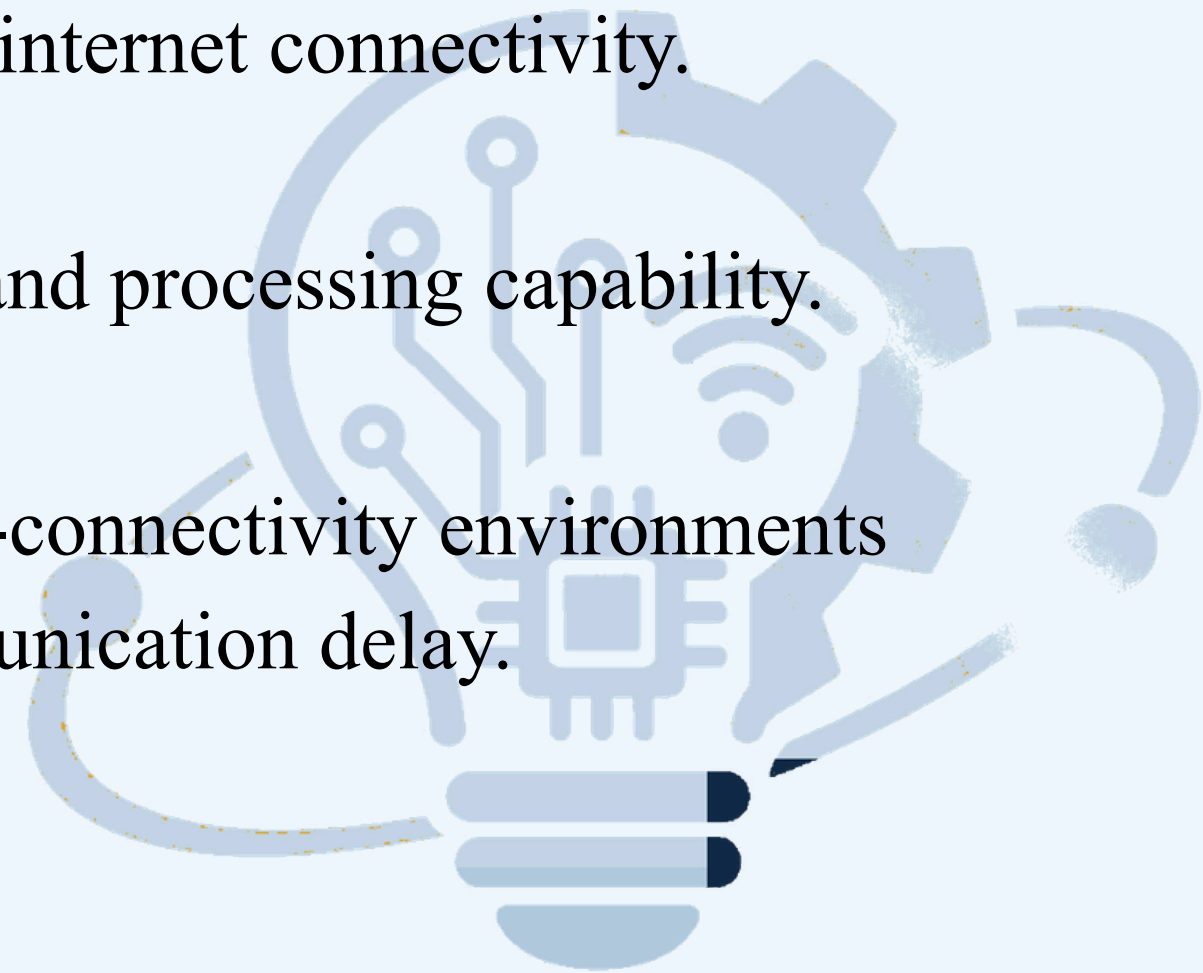
- Most voice assistants depend on cloud processing and internet connectivity.
- Voice data is often transferred to external servers
- Low-powered microcontrollers have limited memory and processing capability.

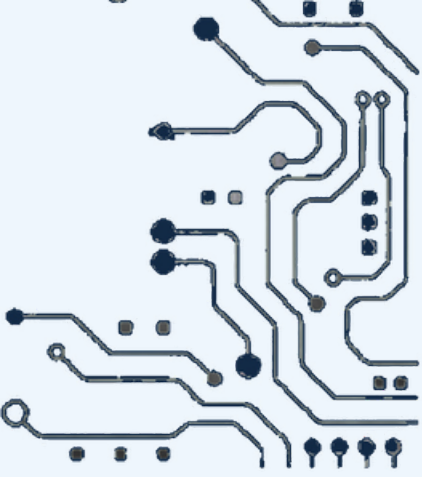
WHY IT MATTERS ?

- Internet-independent voice interaction is useful in low-connectivity environments
- Local processing improves privacy and reduces communication delay.
- Embedded devices need lightweight AI models.

CURRENT CHALLENGES :

- Reliable wake-word detection
- Background noise and false detection
- Limited memory and computational resources





E-TRONIX'26



PROJECT NAME :

Hello Buddy - An Offline TinyML Wake-Word Detection System

PROPOSED SOLUTION :

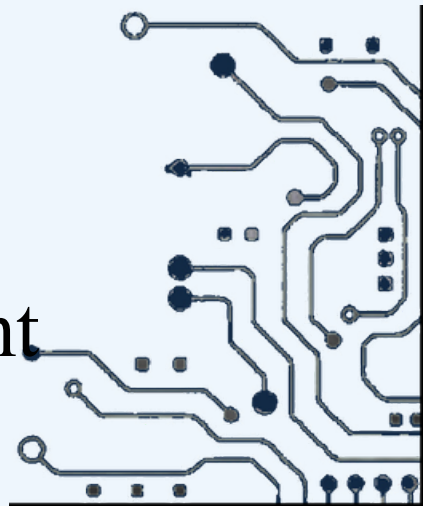
A low-power ESP32-based system that performs real-time, offline wake-word and voice-command detection using MFCC features and TinyML, enabling local decision-making without cloud connectivity.

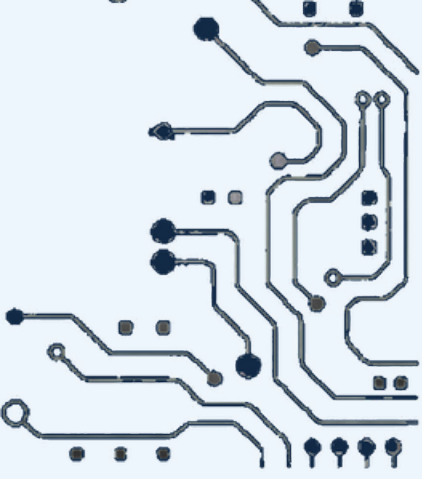
HOW IT SOLVES THE PROBLEM ?

- Offline: Runs entirely on ESP32 without cloud or internet.
- TinyML: Uses lightweight MFCC-based processing for limited MCU resources.
- Noise handling: Trained model improves reliable recognition.
- Low power: Designed for efficient microcontroller operation.

INNOVATION AND UNIQUENESS :

Unlike existing cloud-based solutions, our system enables offline, private, and resource-efficient voice detection directly on a low-power ESP32.

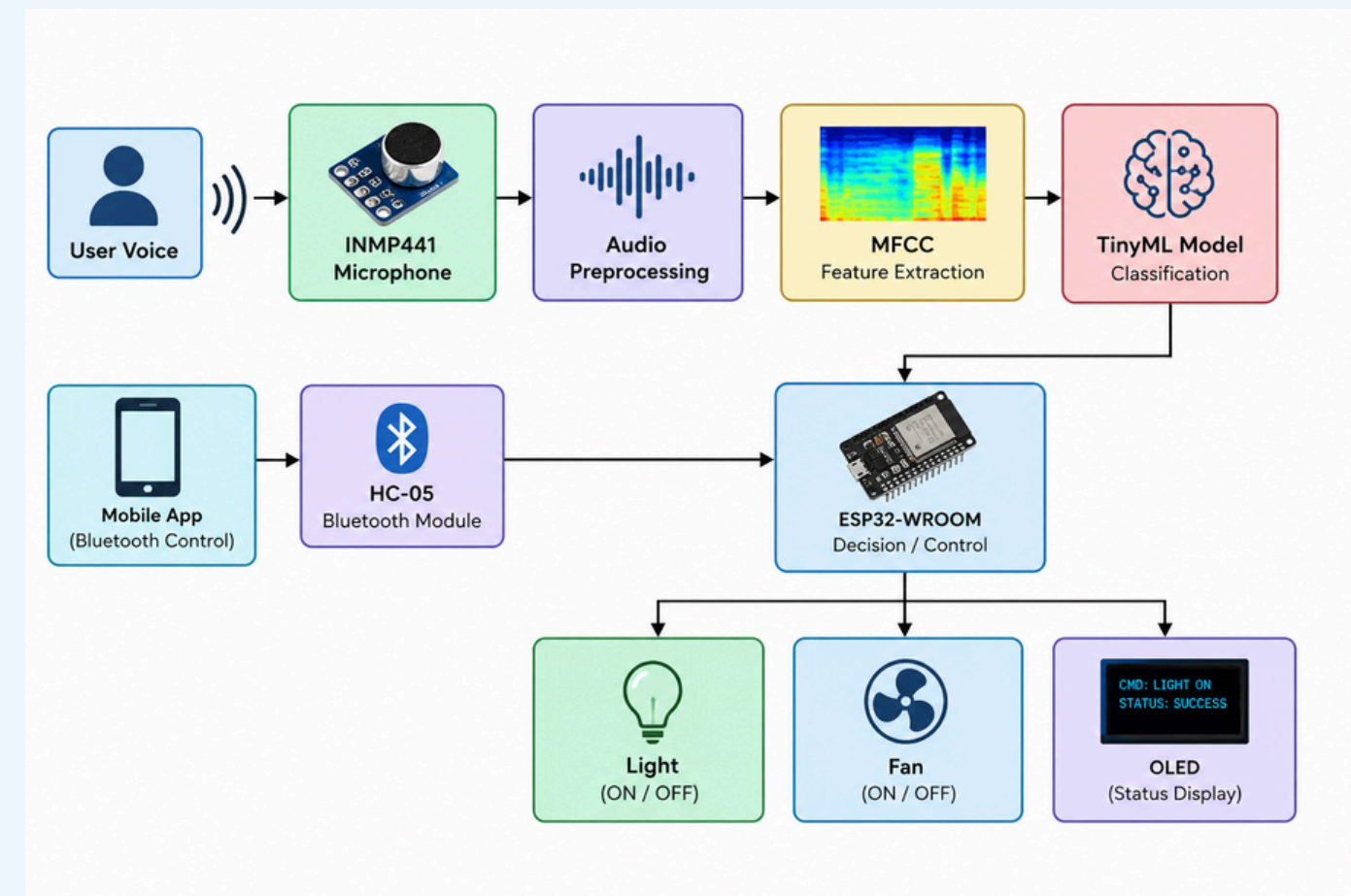




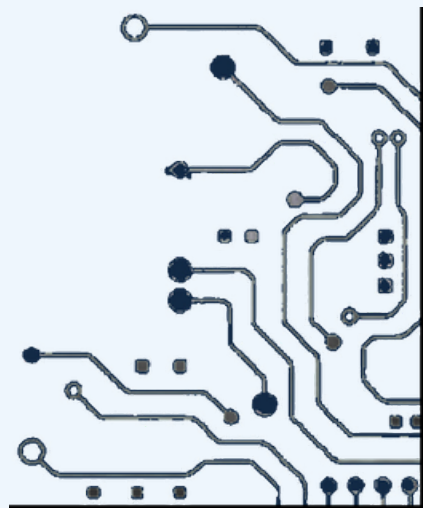
E-TRONIX'26

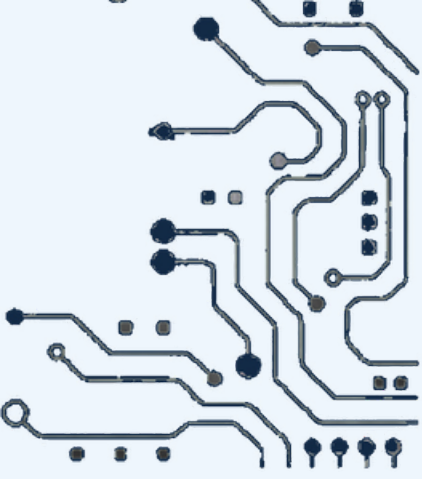


- Voice Input: User speaks the wake word or command, such as “Hello Buddy” or a light/fan command.
- Audio Capture: The INMP441 microphone captures the voice as digital audio.
- Audio Preprocessing: The captured audio is prepared and cleaned for reliable processing.
- MFCC Extraction: Important speech characteristics are extracted using MFCC features.
- TinyML Classification: The TinyML model analyzes the MFCC features and identifies the spoken command.
- ESP32 Decision: The ESP32-WROOM receives the classification result and decides the required action.
- Appliance Control: The ESP32 switches the light or fan ON/OFF according to the detected command.
- Status Display: The OLED displays the current command and device status.



USER VOICE → INMP441 MICROPHONE → AUDIO PREPROCESSING → MFCC FEATURE EXTRACTION → TINYML MODEL CLASSIFICATION → ESP32-WROOM DECISION / CONTROL





E-TRONIX'26



PURPOSE OF BLOCKS :

1. Input Block

- INMP441: Captures voice commands.
- HC-05: Receives mobile commands

2. Processing Block

- Audio Preprocessing: Prepares voice data.
- MFCC: Extracts speech features.
- TinyML: Classifies commands.
- ESP32: Makes control decisions.

3. Output Block

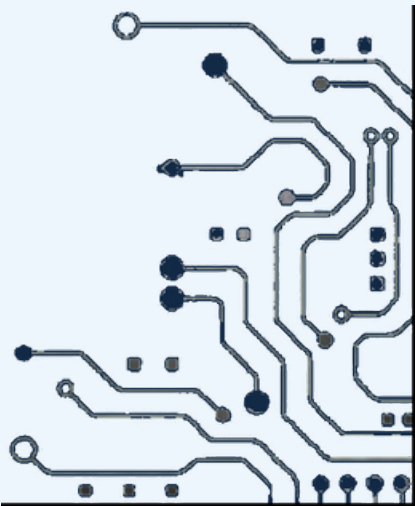
- Light: ON/OFF control.
- Fan: ON/OFF control.
- OLED: Displays status.

IDEA BEHIND CHOOSING EACH BLOCK :

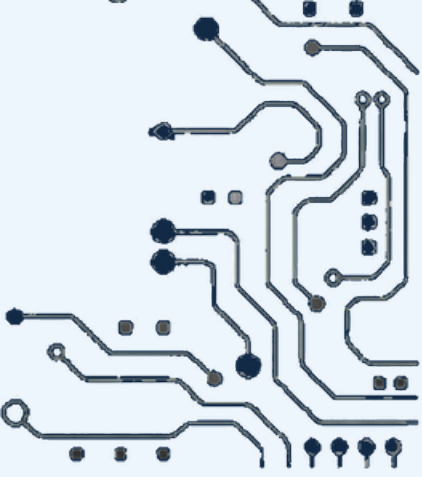
- ESP32: Compact and low-cost processing.
- MFCC + TinyML: Lightweight offline speech recognition.
- HC-05: Enables Bluetooth control.

TRADE-OFFS

- ESP32 has limited resources.
- Voice + Bluetooth adds complexity.
- Offline processing keeps voice data local.

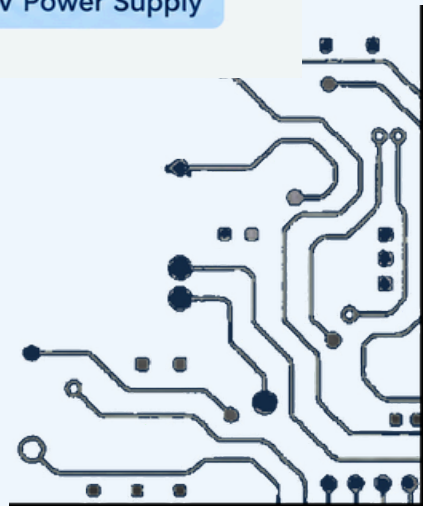
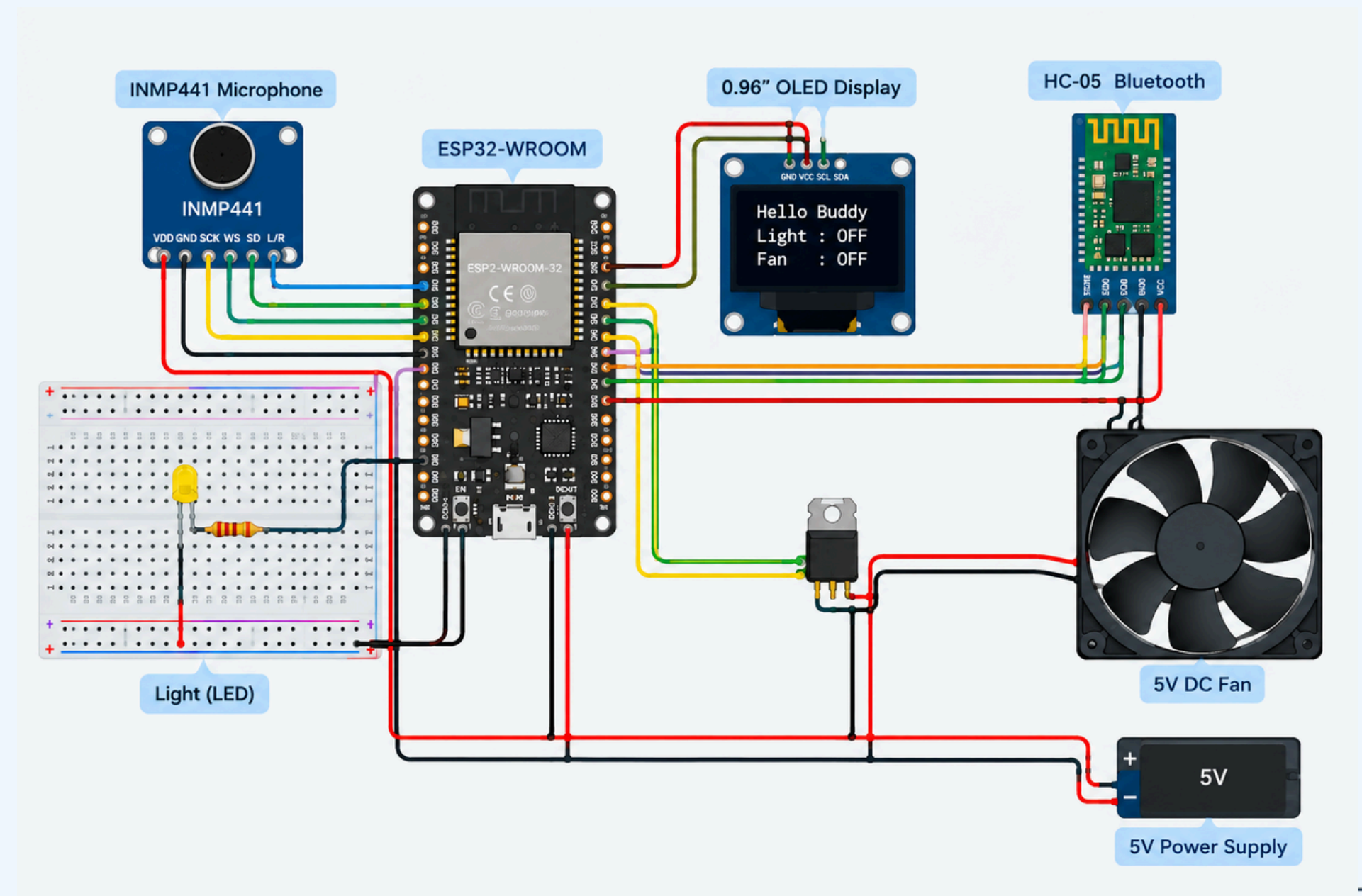


E-TRONIX'26



Outputs and Results :

- Successful offline voice-command recognition.
- Real-time appliance response from detected commands.
- Reduced dependence on internet connectivity.
- Instant status confirmation through OLED.
- Dual control achieved using voice and Bluetooth.
- Functional prototype demonstrated on ESP32-WROOM.
- Lightweight TinyML model successfully deployed on the microcontroller.



E-TRONIX'26



RESEARCH AND REFERENCES

- Edge Impulse – Keyword Spotting / TinyML

<https://docs.edgeimpulse.com/>

- Speech Enhancement for Wake-word Detection

<https://www.espressif.com/en/products/socs/esp32>

